



Unidad 4

Interfaz gráfica y eventos

Prof: Lic. Rosales Pablo (prosales@unpata.edu.ar)
Materia: Programación Orientada a Objetos
Año: 2026





La pre clase, en dos preguntas

- El local quiere una ventana para la caja: un campo, un botón "Registrar" y el total del día. Se escribe todo en una clase, **VentanaCaja**. Funciona.
- El dueño quiere ese total en el resumen por correo. El contador quiere comprobar la suma para diez montos. ¿Dónde está el total?
- ¿Quién llama a **actionPerformed()**? Buscarlo en el código.

El total vive adentro de una ventana, y a **actionPerformed()** no lo llama nadie del programa.



El problema: la ventana hace de todo

```
public class VentanaCaja extends JFrame implements ActionListener {  
    private double total;                                // el negocio  
    public VentanaCaja() {  
        // ... arma los componentes ...                // la pantalla  
        this.botonRegistrar.addActionListener(this);  
    }  
    public void actionPerformed(ActionEvent evento) {  
        double monto = Double.parseDouble(this.campoMonto.getText());  
        this.total += monto;                             // el negocio, otra vez  
        this.etiquetaTotal.setText("Total del día: $" + this.total);  
    }  
}
```

Cómo se ve, qué pasa al presionar y cuánto lleva cobrado el local, en la misma clase.

**La lógica del programa no
tiene que saber que hay una
pantalla.**

**La clase pasada dio el nombre del problema. Hoy, el
mecanismo para separarlo.**



Quién tiene el control

Hasta hoy

- **main** ejecuta la primera instrucción, la segunda, un bucle, y termina.
- El usuario participa cuando el programa se lo pide, con un **Scanner**.
- El programa tiene el control.

Con una ventana

- El programa arma la pantalla, la muestra, y espera.
- Cada click es un evento. El programa reacciona.
- **main** termina enseguida; Swing deja un hilo esperando eventos.
- El usuario tiene el control.

En una interfaz gráfica el programa no ejecuta, espera. El usuario decide qué corre y cuándo.



Los componentes de Swing

- **JFrame**, la ventana. Título, tamaño, y adentro los demás.
- **JLabel**, un texto que el usuario no modifica.
- **TextField**, un campo de una línea. Lo escrito se lee con `getText()`, y siempre es un **String**.
- **Button**, el botón. Cuando se presiona, avisa.
- Crear, ubicar con `setBounds(x, y, ancho, alto)`, agregar con `add()`. Ventana de tamaño fijo con `setResizable(false)`. Y al final `setVisible(true)`.

Una ventana es un contenedor con componentes adentro. Hasta `setVisible(true)`, no existe para el usuario.



Cómo avisa un botón

- El botón no sabe qué hacer. El mismo **JButton** registra un pago, convierte moneda o cierra sesión.
- **Fuente:** el componente donde ocurre la acción, el botón.
- **Evento:** el objeto que Swing crea para describir qué pasó, un **ActionEvent**.
- **Receptor:** el objeto anotado para que le avisen. Implementa **ActionListener**, un solo método, **actionPerformed()**.
- Se anota con **boton.addActionListener(receptor)**. A eso se lo llama delegación de eventos.

El botón no sabe qué hacer, solo a quién avisar.



Cómo avisa un botón

```
public interface ActionListener {                                // java.awt.event
    void actionPerformed(ActionEvent evento);
}
public class ControladorCaja implements ActionListener {
    public ControladorCaja(VistaCaja vista, Caja caja) {
        vista.getBotonRegistrar().addActionListener(this);    // registro
    }

    public void actionPerformed(ActionEvent evento) {
        // corre con cada click. Nadie del programa lo llama: lo llama
        Swing.
    }
}
```

JButton no puede depender de ControladorCaja. Define una interfaz y el código nuevo se adapta. Es Dependency Inversion en la biblioteca.



Tres clases en vez de una

Tres responsabilidades

- Cómo se ve la pantalla. La pide el cajero.
- Cuánto lleva cobrado el local. La pide el dueño, y es la misma cuenta del cierre.
- Qué pasa al presionar el botón. Conecta a las otras dos.

Tres clases

- **Modelo:** `Caja`. Sabe del negocio. No importa `javax.swing`.
- **Vista:** `VistaCaja`. Arma la ventana y sabe mostrar. No hace cuentas.
- **Controlador:** `ControladorCaja`. Implementa `ActionListener` y traduce entre las dos.

El modelo no importa `javax.swing`. Esa es la prueba de que la lógica no sabe que hay una pantalla.



La vista

```
public class VistaCaja {  
    private final JFrame ventana;           // tiene una ventana, no es una  
    private final JTextField campoMonto;  
    private final JLabel etiquetaTotal;  
    private final JButton botonRegistrar;  
    public void mostrar() { this.ventana.setVisible(true); }  
    public JButton getBotonRegistrar() { return this.botonRegistrar; }  
    public String getTextoMonto() { return this.campoMonto.getText(); }  
    public void mostrarTotal(double total) {  
        this.etiquetaTotal.setText(String.format("Total: $%.2f", total));  
    }  
}
```

Expone el botón, porque el controlador tiene que registrarse. El campo y la etiqueta quedan adentro.

El controlador

```
// ControladorCaja implements ActionListener
public void actionPerformed(ActionEvent evento) {
    try {
        double monto = Double.parseDouble(this.vista.getTextoMonto());
        this.caja.registrar(new PagoEfectivo(monto));
        this.vista.mostrarTotal(this.caja.total());
    } catch (NumberFormatException ex) {
        this.vista.mostrarError("Ingrese un numero");
    } catch (IllegalArgumentException ex) {
        this.vista.mostrarError(ex.getMessage()); // lo que rechazo el
modelo
    }
}
```

Traduce en las dos direcciones: de la pantalla al modelo, y del modelo a la pantalla.



Y main

```
public static void main(String[] args) {  
    Caja caja = new Caja(new ServicioCorreo("smtp.lapampa.com", 587));  
    VistaCaja vista = new VistaCaja();  
    ControladorCaja controlador = new ControladorCaja(vista, caja);  
    vista.mostrar();  
}  
// main termina aca. La ventana sigue: Swing dejo un hilo esperando eventos.
```

Los new de las tres clases en un solo lugar. Es la inyección de dependencias de la clase pasada, y la Caja es la misma.



Una carpeta por responsabilidad

Paquetes

- `src/modelo`, `src/vista`, `src/controlador`. Programa en la raíz.
- Primera línea de cada archivo: `package modelo;`. La carpeta y el nombre tienen que coincidir.
- Quien usa una clase de otro paquete la importa: `import`

La regla de la clase, verificable leyendo tres líneas de import.

- Sin `public`, la clase solo se ve desde su paquete.

Lo que se ve

- `modelo` no importa `vista`, ni `controlador`, ni `javax.swing`.
- `controlador` importa a los otros dos. Es el único que los conoce.
- `Programa` importa lo que necesita para armar los tres.



Qué se ganó

- El total del correo sale de `caja.total()`, el mismo objeto que usa la ventana.
- Comprobar la suma para diez montos es un `main` de diez líneas, sin ventana.
- El botón a la derecha o la letra más grande tocan `VistaCaja` y nada más.
- A este reparto se lo llama Modelo-Vista-Controlador. Aparece, con variantes, en casi todas las bibliotecas gráficas y los frameworks web.

Modelo, vista y controlador son tres responsabilidades, y tres personas distintas que piden cambios.



Tres preguntas para saber si está bien repartido

- ¿El paquete `modelo` importa algo de `javax.swing`, de `vista` o de `controlador`? No debería.
- ¿La vista hace alguna cuenta o toma alguna decisión del negocio? No debería.
- ¿Se puede probar el modelo desde un `main` sin abrir una ventana? Debería.

Basta con mirar los `import`.



La actividad de hoy

- Un conversor de pesos a dólares en una sola clase, **VentanaConversor**, que es la ventana, atiende el click y hace la cuenta. Con -5 muestra un resultado.
- **Parte 1:** buscar quién llama a **actionPerformed()**, contar responsabilidades, y decidir si un conversor es una ventana.
- **Parte 2:** **modelo/Conversor**, **vista/VistaConversor** y **controlador/ControladorConversor**. El proyecto compila en todo momento; se cambia de una versión a la otra en **Programa**.
- **Parte 3:** probar el modelo sin ventana, y agregar los euros. Un método en el modelo, un botón en la vista, un **if** en el controlador.

En el proyecto integrador

El juego sigue funcionando sin pantalla. La pantalla es una vista más, y la partida no importa `javax.swing`.
